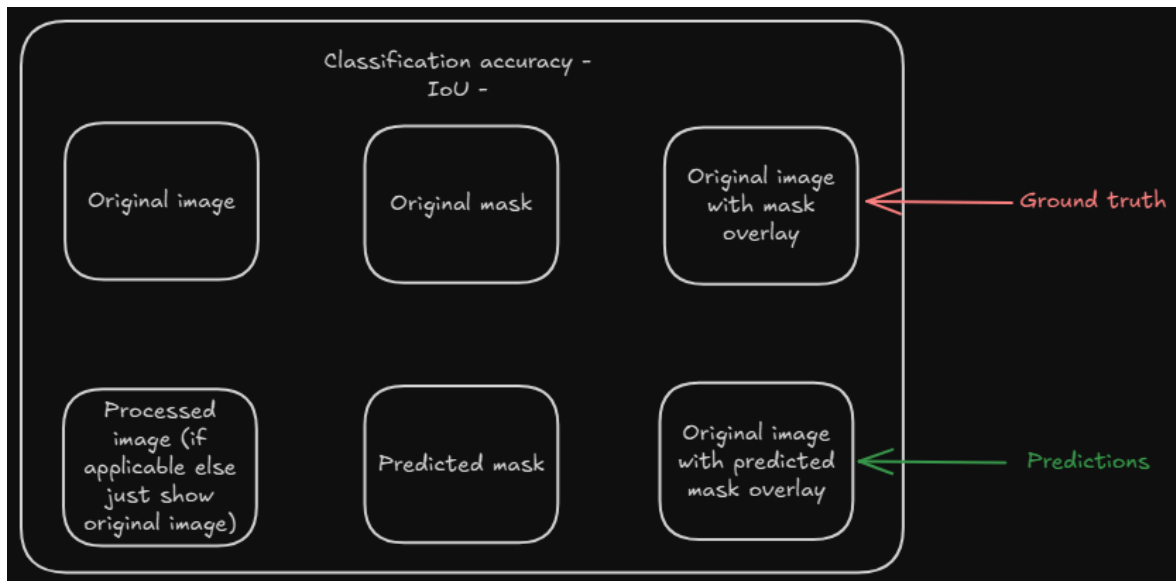


Dataset link - <https://www.kaggle.com/datasets/briscdataset/brisc2025?resource=download>

Minimum expectations -

1. Prepare a **U-Net** architecture (<https://arxiv.org/abs/1505.04597>) to segment the data.
2. Attach a classifier head to the encoder output and classify the data.
3. Prepare a code that takes any image as an input and displays the image in the form -



4. Update the **U-Net** architecture to **Attention U-Net** (<https://arxiv.org/abs/1804.03999>) and run the segmentation training again.

Open-ended design choices -

- You can train the segmentation and classification heads separately or together. If you opt to do both and show and analyze differences of training separately and training together, you will get some bonus marks.
- You do not have to separately preprocess each image, you can find a common pattern and apply a preprocessing pipeline. It's your duty to ensure you are getting the best possible results with or without preprocessing.
- You can use any classifier architecture. There are no restrictions on which one to use. Whatever you do, try to maximize your accuracy.
- You do not have to make a presentation slide if you opt not to. But in that case, you have to prepare the notebook carefully so that you can present your work properly.

Bonus tasks -

Bonus marks will only be awarded at the discretion of the lab faculties. We are providing a list of possible cases where bonus marks may be provided. But remember, if you do not complete the

bonus tasks properly, you will not receive any additional marks. Incomplete tasks are worth 0 marks. With that being said, the list of bonus tasks that you may attempt (you can go for multiple bonus tasks) (the task values are arranged in ascending order) -

1. From the section “Open-ended design choices”, analyze how the performance changes when the segmentation head and the classifier head are trained separately compared to when they are trained together. (Easy difficulty)
2. Test out different well-established classifier architectures and see how the performance changes. (Easy difficulty)
3. Test how your model performance changes with change in hyperparameters. For example, you can test for the most optimal optimizer under a wide range of learning rates. Remember, you have to compile your results properly for your work to be considered complete. (Medium difficulty)
4. Implementing EfficientDet (<https://arxiv.org/abs/1911.09070>) architecture on the decoder side of the original U-Net architecture. (High difficulty)

If you want to do anything different for the bonus segment, something novel, do let your lab faculties know and get it approved before working on it.